

Three detectors on the same traffic: a comparison of supervised and unsupervised intrusion detection on CICIDS2017

Youssef Amar

github.com/2Fick/AI-Powered-IDS

September 6, 2026

Abstract

A random forest, an isolation forest and a dense autoencoder are trained on the CICIDS2017 network capture and served behind one API that scores flows one at a time. The supervised model reaches 99.87 % recall at a 0.07 % false positive rate and decides in 1.66 ms, and this report is mostly about why that number should not be read on its own. Three results qualify it. A random train and test split leaves 27.8 % of the test attacks with a byte identical twin in the training half, though a time ordered split shows the supervised model survives this while the unsupervised ones lose half their recall. Hyperparameter sweeps show the forest reaches its recall at five trees and that the scikit-learn default for the isolation forest sits near the worst end of its own curve. Removing one attack family from training at a time shows the supervised model does not degrade gracefully on an unseen family, it collapses to zero on three of six, and that the unsupervised models recover a third of the loss on two families and nothing on the rest.

1 Introduction

Intrusion detection on CICIDS2017 is a solved problem by the usual measure. Published results routinely report accuracy above 99 %, and a random forest fitted in a few minutes reproduces them. That makes the headline number an uninteresting thing to report and an easy thing to overstate.

This project therefore compares three detectors that answer the same question in different ways, and spends most of its effort on what qualifies the result rather than on the result itself.

- A **random forest**, supervised, learns from labelled attacks.
- An **isolation forest**, unsupervised, is fitted on benign traffic and scores how easily a flow is isolated from the rest.
- An **autoencoder**, unsupervised, is fitted on benign traffic and scores how badly it fails to rebuild a flow.

All three are served by one FastAPI application, and a dashboard replays held out traffic through them at a controlled rate so that latency and detection can be watched rather than quoted.

2 Data and preprocessing

2.1 Which release, and why

CICIDS2017 is five working days of traffic from a test bed at the University of New Brunswick, captured in July 2017 with real attacks run against real machines. It ships in two forms. The widely used *machine learning* CSVs keep 78 flow statistics and the label, and drop the addresses. The *labelled flow* release keeps the same statistics and adds flow identity: source and destination address, ports, protocol and timestamp.

This project uses the second. Without addresses the threat intelligence enrichment would have nothing real to look up, and without timestamps there is no capture order to replay. The feature set handed to the models is unchanged.

2.2 What the raw files actually contain

Reading the eight capture files before modelling anything turned up four problems that each changed a decision downstream.

Padding rows. The Thursday morning file carries 288,602 rows in which every single column is empty. They have no label, so a naive reading counts all of them as attacks, which would have shifted the class balance by ten percentage points.

Infinities. 4,376 values are infinite, all in **Flow Bytes/s** and **Flow Packets/s**, produced when a flow lasts zero microseconds and the generator divides by it.

Dead columns. Eight of the 77 numeric columns hold the same value on every flow in the dataset. They cost memory and carry no signal.

Non ASCII labels. The three web attack labels are written with a Windows code page dash that breaks any terminal or file that is not cp1252, and makes an awkward JSON key.

2.3 Class balance

Figure 1 is the reason accuracy is never quoted in this report. Benign traffic is 80.3 % of the dataset, so a model that never raises an alert already scores above eighty. Within the attacks the imbalance is far worse: DoS Hulk has 231,073 flows and Heartbleed has 11, a ratio above twenty thousand to one. Recall per family, not one aggregate number, is the only honest summary.

2.4 Cleaning and splitting

Cleaning drops the padding rows, the infinities, 115 flows with a negative duration and the eight dead columns, leaving 2,827,761 flows and 69 features. Only **Destination Port** is kept from the identity columns, which matches the standard feature set; the rest travel alongside for display and enrichment but are never given to a model.

A 60,000 flow replay slice is taken out first, before the train and test split, so nothing the dashboard streams was ever seen during fitting. The rest is split 75 to 25, stratified on the attack family so the rare ones survive on both sides. All three splits hold 19.68 % attacks.

The replay slice is shuffled rather than ordered by capture time, and Figure 2 is why. Monday is benign only and accounts for a fifth of the data, so a chronological replay spends its first quarter with nothing to detect, while Wednesday and both Friday afternoon sessions are more than a third attacks. Shuffling gives every second of the stream the attack density of the capture as a whole. Chronological order remains available behind a flag.

2.5 Scaling

Figure 3 explains the one preprocessing decision that is not routine. The largest value of **Total Length of Bwd Packets** sits 9,125 times above its own 99th percentile, and nine other features are in the same condition. Standardising such a column leaves a handful of flows setting the scale for all of it, and the autoencoder loss becomes a report on those few flows.

A quantile transform fixes this and was the first choice. It also cost 20 ms to transform a single row, which was more than all three models spent deciding put together. The final transform is $\text{sign}(x) \log(1 + |x|)$ followed by standardisation: it compresses the same tails, runs in microseconds, and is a plain array expression rather than a library call with per invocation validation.

3 Method

3.1 Fitting

The random forest is fitted on the full training split with balanced subsample class weights, and alerts above a probability of 0.5.

The two unsupervised models never see an attack. Both are fitted on the 1,667,258 benign flows of the training split, and both take their threshold from the same rule: the quantile of their own score distribution on that benign data corresponding to a 1 % target false positive rate. Without a shared rule they would be compared at whatever operating point their library happened to default to.

3.2 Serving

All three sit behind one interface, so a number in the benchmark is produced by the same code path that serves a live prediction. Two details matter for latency. The tree ensembles drop to a single worker at serving time, because handing one row to a thread pool costs more than scoring it; this alone took the forest from

17 ms to 3 ms per flow. And the scaler, described above, accounts for the rest.

4 Hyperparameter study

4.1 How many trees

Figure 4 shows recall between 99.76 % and 99.78 % from five trees to two hundred, while the time to score one flow rises from 0.33 ms to 6.70 ms. Every tree past the first handful buys nothing and costs latency. The deployed forest was reduced from 100 trees to 50, which halved the served latency from 3.09 ms to 1.66 ms with no measurable change in recall.

4.2 How much each isolation tree sees

Figure 5 shows recall climbing monotonically with the number of flows each tree is fitted on, from 0.95 % at 128 samples to 17.0 % at 16,384. The scikit-learn default of 256 sits near the worst end of that curve. Moving to 16,384 took recall on the full test split from 11.3 % to 17.2 % at the same false positive rate.

4.3 When to stop training the autoencoder

Figure 6 is the argument for measuring detection quality every epoch rather than only the loss. Over 25 epochs the reconstruction loss falls steadily from 0.540 to 0.036, but the area under the ROC curve peaks at epoch 7 at 0.932 and then drifts back to 0.918, while recall keeps climbing until roughly epoch 16 and flattens. Training longer makes the reconstruction better without making the detector better.

5 Results

Table 1 reports the three models on 691,941 held out flows, 136,187 of them attacks.

Figure 7 is the more informative view. The supervised model is near perfect on every family with more than a thousand examples and drops to 81 % on Bot and 67 % on Infiltration, the two rarest. Those are the only two places where an unsupervised model competes: the autoencoder reaches 89 % on Infiltration. Neither unsupervised model detects a port scan at all, because a port scan is a small, ordinary looking flow, and asking whether a flow is unusual in general is not the same question as asking whether it is unusual for its destination port.

Figure 8 shows the whole trade off rather than one point on it, and Figure 9 shows why the autoencoder cannot do better at any threshold: the benign and attack score distributions overlap heavily, and the threshold is placed where it is only because the false positive budget says so.

6 How much to trust the numbers

6.1 Leakage under a random split

A denial of service burst produces thousands of flows identical in every feature, so a random split puts copies of the same row on both sides. The dataset holds 2,497,153 distinct feature vectors for 2,827,761 flows; 14.1 % of a random test split has a byte identical twin in the training half, and 27.8 % of the test attacks do.

Refitting under a split that trains on the earlier part of every capture session and tests on the later part gives Table 2. The supervised model barely moves and its false positive rate improves, so the duplicates are real but they are not what produces the headline number. The unsupervised models lose half their recall or more, and that is the useful finding: their thresholds are fitted on the benign traffic of the training window, and benign traffic drifts within a working day. Anything deployed this way needs its threshold recalibrated on a rolling window rather than fixed once at training time.

6.2 Attacks nobody trained on

The supervised model works under a closed world assumption. To measure what that costs, one family at a time was removed from training, all three models refitted, and each asked about the family none of them was shown.

Figure 10 did not come out the way an argument for an ensemble usually goes, and the result is more useful than that argument would have been. The supervised model does not degrade on an unseen family, it collapses on three of six and holds up on the other three, and the pattern is whether a structurally similar family remains in training. DDoS survives at 63.9% because DoS Hulk is still there; slowloris survives at 89.8% because the other slow denial of service families are; web brute force survives at 80.4% because cross site scripting and SQL injection are. PortScan, Bot and FTP-Patator have no close relative left and the model goes to zero on all three.

The unsupervised models are not a safety net either. They recover 33.7% on DDoS and 32.2% on slowloris, and nothing at all on the other four.

7 Conclusion

The supervised model is the one to deploy. It is accurate, it survives a harder split, and at 1.66 ms per flow it keeps up with a respectable link on one core. The two unsupervised models earn a narrower place: they help on the rarest families, where labels are too few for supervision to work, and they recover part of the loss on two of six unseen families.

What none of them does is handle a genuinely new attack. A system built on these needs a retraining pipeline and human review, not a detector assumed to catch the unknown. Three further directions follow from the measurements above rather than from a wish list: recalibrating the unsupervised thresholds on a rolling window, weighting the autoencoder reconstruction error per feature so that an anomaly confined to one dimension is not averaged away across 69, and conditioning the models on destination port so that “unusual” can mean “unusual for this service”.

A Tables

Table 1: Held out test split. Both unsupervised models are thresholded at a 1 % target false positive rate.

Model	Recall	FP rate	AUC	Latency
Random Forest	99.87 %	0.07 %	1.000	1.66 ms
Isolation Forest	17.24 %	1.01 %	0.921	3.29 ms
Autoencoder	37.69 %	1.01 %	0.930	0.33 ms

Table 2: Recall under a random split and under a time ordered split.

Model	Random	Time ordered
Random Forest	99.9 %	98.5 %
Isolation Forest	6.9 %	1.8 %
Autoencoder	43.2 %	22.8 %

B Figures

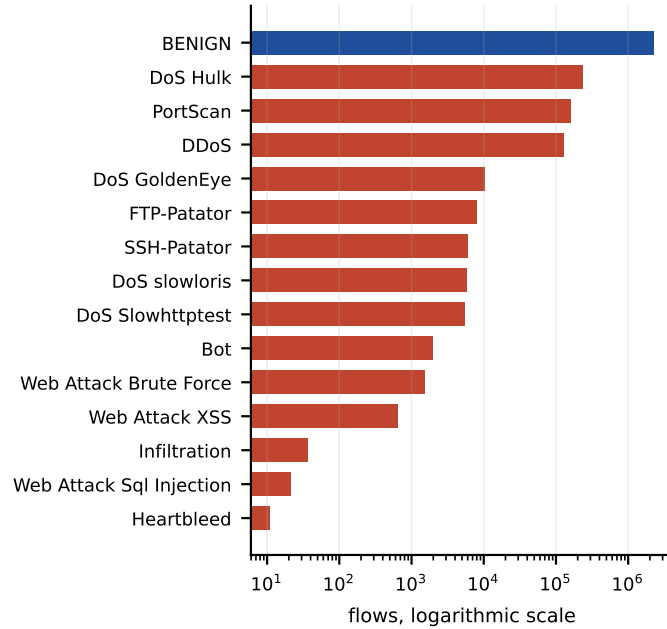


Figure 1: Flows per class on a logarithmic axis. Benign in blue, attacks in red. On a linear axis every family below the top three is invisible.

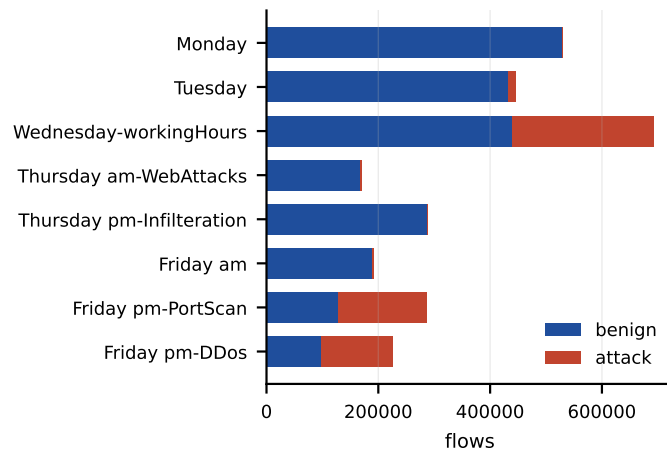


Figure 2: Benign and attack traffic per capture session. Monday is benign only and the attack share elsewhere swings from 0.01 % to 57 %.

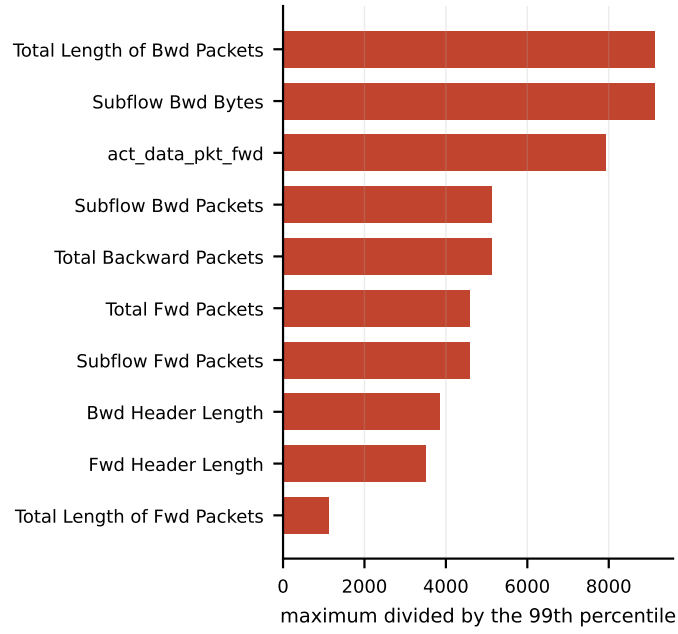


Figure 3: Maximum value divided by the 99th percentile, for the ten heaviest tailed features.

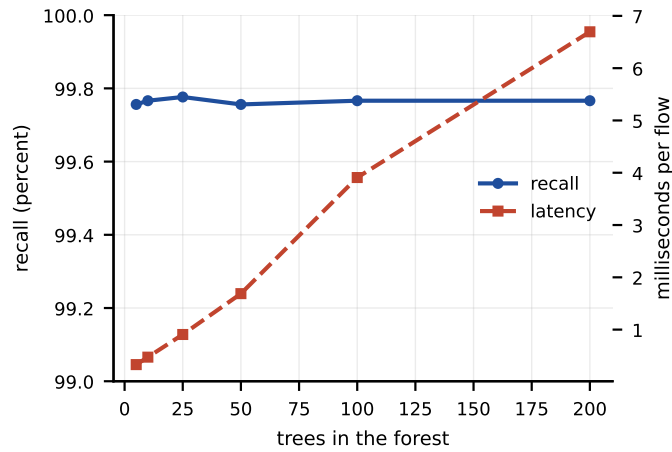


Figure 4: Random forest recall and per flow latency against the number of trees. The two lines do not agree.

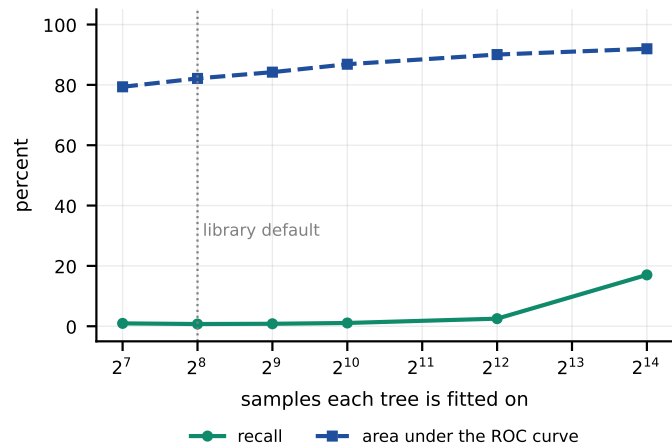


Figure 5: Isolation forest quality against samples per tree. The dotted line marks the library default.

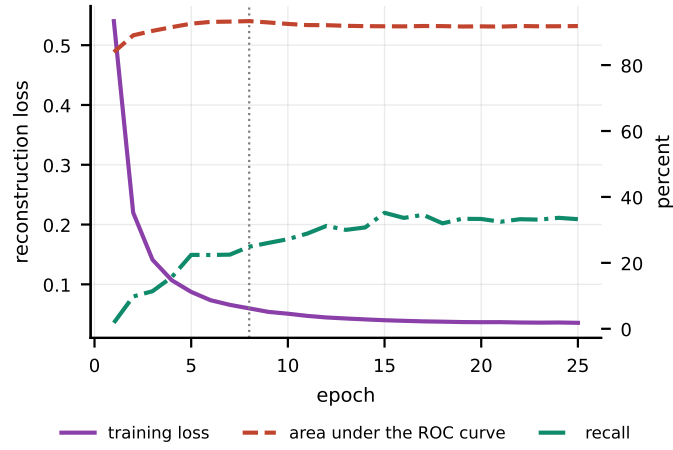


Figure 6: Autoencoder loss, area under the ROC curve and recall by epoch. The dotted line marks the epoch of best ranking quality.

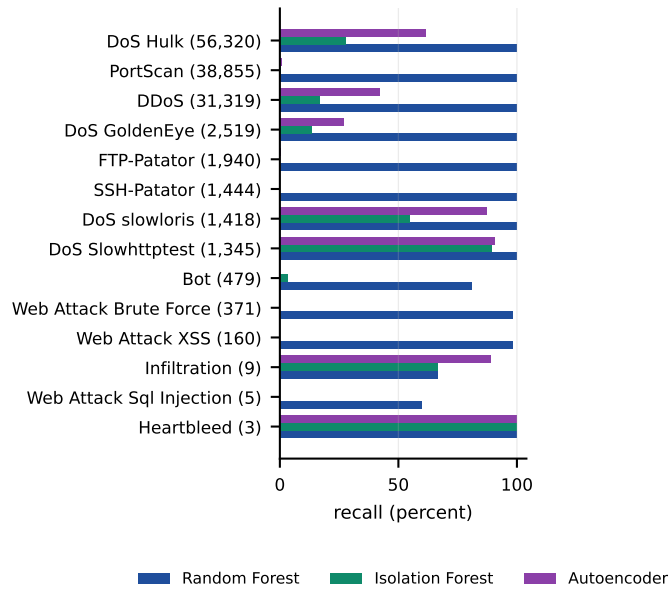


Figure 7: Recall per attack family, with the number of test flows in parentheses. Families are ordered by size.

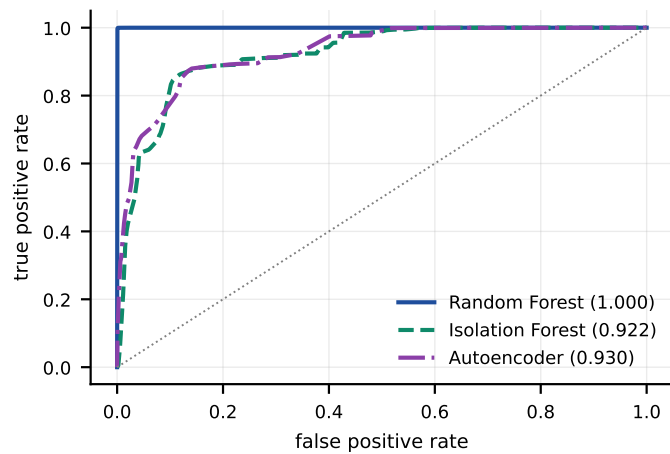


Figure 8: ROC curves with the area under each in parentheses.

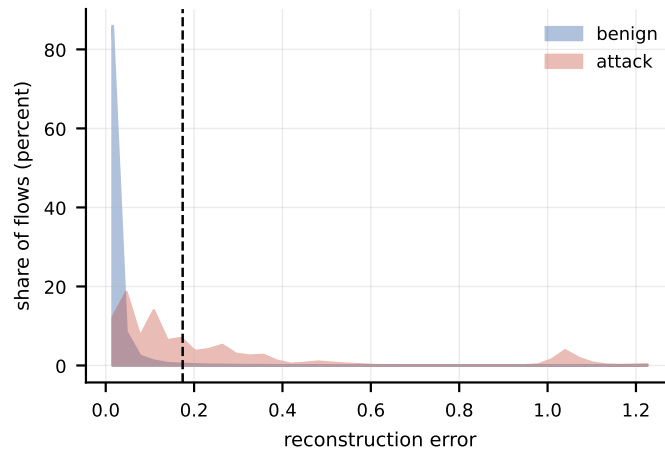


Figure 9: Autoencoder reconstruction error for benign and attack flows, each normalised to its own class. The dashed line is the threshold in use.

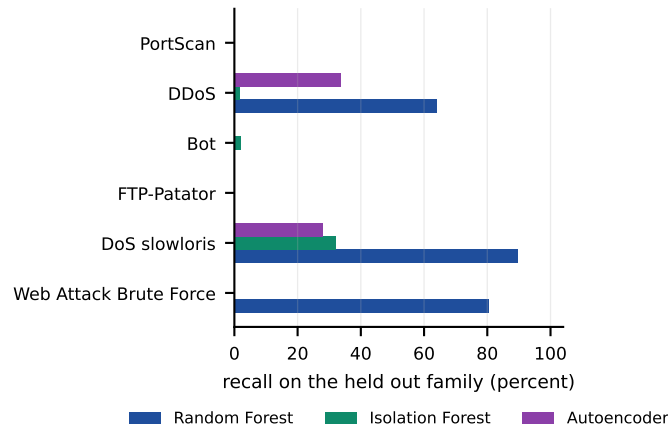


Figure 10: Recall on a family removed from training. The unsupervised models see no attacks during training, so only the supervised one changes.

Reproducing this report

Every figure and every number above comes from a command in the repository.

```
python -m ids.data.download
python -m ids.data.explore --out reports
python -m ids.data.preprocess
python -m ids.train
python -m ids.benchmark
python -m ids.sweep
python -m ids.curves
python -m ids.validate
python -m ids.novelty
python -m ids.report.figures
```

Dataset: I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*, ICISSP 2018.